Stony Brook University

# CSE 508

## *Network Security (Lecture 4- Public Key Encryption and Digital Signatures)*

**Omar Haider Chowdhury**
*Empire Innovation Scholar and Associate Professor*
*Department of Computer Science*
*Stony Brook University*

# Modules

Stony Brook University

- This class will have the following modules:
  - Background:
    - Cryptography
    - Introduction to Network and Protocols
  - Authentication
  - Access Control
  - Attacks and Defenses against protocols
  - Essential tools for attack and defense
  - Web Security
  - Privacy and Anonymity
  - Advanced Topics (Protocol Verification)

# Modules

Stony Brook University

- This class will have the following modules:
  - Background:
    - Cryptography
    - Introduction to Network and Protocols
  - Authentication
  - Access Control
  - Attacks and Defenses against protocols
  - Essential tools for attack and defense
  - Web Security
  - Privacy and Anonymity
  - Advanced Topics (Protocol Verification)

# Review of Symmetric Key Cryptography

# Symmetric Key Cryptography

Stony Brook University

- **Confidentiality:**
  - Stream cipher (uses PRNG)
  - Block ciphers with encryption modes (CBC or CTR)
- **Integrity:**
  - Cryptographic hash functions
  - Message authentication code (Keyed Hash Functions)
- **Limitation:** *Sender and Receiver must share the same key*
  - Needs secure channel for key distribution
  - Impossible for two parties having no prior relationship
  - Needs many keys (i.e., $n^2$) for $n$ parties to communicate

# Public Key Cryptography

# Concept of Public Key Encryption

Stony Brook University

- **Each party has a pair ($K$, $K^{-1}$) of keys**:
  - $K$ is the **public** key, and used for encryption
  - $K^{-1}$ is the **private** key, and used for decryption
  - Satisfies $\mathbf{D}_{K^{-1}}[\mathbf{E}_K[M]] = M$
- **Knowing the public-key $K$, it is computationally infeasible to compute the private key $K^{-1}$**
  - How to check ($K$,$K^{-1}$) is a pair?
  - Offers only computational security. Secure Public Key encryption is impossible when P=NP, as deriving $K^{-1}$ from $K$ is in NP.
- **The public key $K$ may be made publicly available, e.g., in a publicly available directory**
  - Many can encrypt, only one can decrypt

# Concept of Public Key Encryption

Stony Brook University

- **Each party has a pair ($K$, $K^{-1}$) of keys**:
  - $K$ is the **public** key, and used for encryption
  - $K^{-1}$ is the **private** key, and used for decryption
  - Satisfies $\mathbf{D}_{K^{-1}}[\mathbf{E}_K[M]] = M$
- **Knowing the public-key $K$, it is computationally infeasible to compute the private key $K^{-1}$**
  - How to check ($K$,$K^{-1}$) is a pair?
  - Offers only computational security. Secure Public Key encryption is impossible when P=NP, as deriving $K^{-1}$ from $K$ is in NP.
- **The public key $K$ may be made publicly available, e.g., in a publicly available directory**
  - Many can encrypt, only one can decrypt

**Public-key systems aka *asymmetric* crypto systems**

# Early History

Stony Brook University

- Proposed by Diffie and Hellman, documented in "***New Directions in Cryptography***" (1976)
  1. Public-key encryption schemes
  2. Key distribution systems
     - Diffie-Hellman key agreement protocol
  3. Digital signature

- Public-key encryption was proposed in 1970 in a classified paper by James Ellis
  - The paper was made public in 1997 by the British Governmental Communications Headquarters

- **The concept of digital signature is still originally due to Diffie & Hellman**

# Public Key Encryption Algorithms

Stony Brook University

- Most public-key encryption algorithms use either modular arithmetic number theory, or elliptic curves
- ***RSA***
  - Based on the hardness of **factoring large numbers**
- ***El Gamal***
  - Based on the hardness of solving **discrete logarithm problem**
  - Use the same idea as Diffie-Hellman key agreement

# Public Key Encryption Algorithms

Stony Brook University

- Most public-key encryption algorithms use either modular arithmetic number theory, or elliptic curves
- ***RSA***
  - Based on the hardness of **factoring large numbers**
- ***El Gamal***
  - Based on the hardness of solving **discrete logarithm problem**
  - Use the same idea as Diffie-Hellman key agreement

In the computational security model, we prove security of a scheme *S* in the following way:

***If Problem P is hard to solve, then S is secure***

# Preliminary: Modular Arithmetic

Stony Brook University

**Notation:** For any $a$, prime $P$, and $b$, if we can write $a = P \times k + b$ for some $k$ where $b < P$, then alternatively one can write the following: $\boxed{a \equiv b (\text{mod } P)}$.

Given a positive integer $P$, we write $\mathbb{Z}_P$ to denote the following set of integers $\{0, 1, 2, 3, \ldots, P-1\}$.

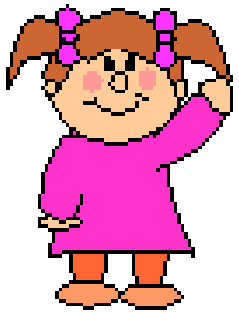
**Useful Formulations**

- $(a + b) \text{ mod } P = ((a \text{ mod } P) + (b \text{ mod } P)) \text{ mod } P$
- $(a - b) \text{ mod } P = ((a \text{ mod } P) - (b \text{ mod } P)) \text{ mod } P$
- $(a * b) \text{ mod } P = ((a \text{ mod } P) * (b \text{ mod } P)) \text{ mod } P$
- $(g^a \text{ mod } P)^b = (g^b \text{ mod } P)^a = (g^{ab} \text{ mod } P)$

# Diffie-Hellman Key Agreement

- **Not a Public Key Encryption system**
- It can allow A and B to agree on a shared secret in a public channel (against passive, i.e., ***eavesdropping only adversaries***)

**Setup**: p prime and **g generator of $Z_p$***, p and g public.



$g^a \bmod p$ →

← $g^b \bmod p$

Pick random, secret a

Compute and send $g^a \bmod p$

$K = (g^b \bmod p)^a = g^{ab} \bmod p$

Pick random, secret b

Compute and send $g^b \bmod p$

$K = (g^a \bmod p)^b = g^{ab} \bmod p$

# Preliminary: Group, Generator

Stony Brook University

- A *group* is a set $G$ together with a binary operation on $G$, denoted here by $\otimes$

- A group has the following three properties:

  1. **Associativity**: $\forall a, b, c \in G.\ (a \otimes b) \otimes c = a \otimes (b \otimes c)$

  2. **Identity element**: $\exists e \in G. \forall a \in G.\ e \otimes a = a$ and $a \otimes e = a$.

  3. **Inverse element**: $\forall a \in G. \exists b \in G.\ a \otimes b = e$ and $b \otimes a = e$.

- **Example**: Addition over integers

  - $\otimes = +$
  - **Identity element** is 0.
  - **Inverse element** of $x$ is $-x$.

# Preliminary: Group, Generator (contd)



- A *abelian group* is a group $(G, \otimes)$ with the following commutativity property (in addition to the previous three properties of a group)

  1. **Commutivity**: $\forall a, b \in G.\ \ a \otimes b = b \otimes a$.

- **Example of abelian group**: Multiplicative group of integers modulo $P$

  1. $G = \mathbb{Z}_P$ for a given $P$ (Recall that, $Z_P = \{0, 1, 2, 3, \ldots P-1\}$)
  2. $\forall a, b \in G.\ \ a \otimes b = (a \times b)(\text{mod } P)$
  3. When $P$ is drawn from the set $\{1, 2, 4, q^k, 2q^k\}$ where $q$ is an odd prime and $k > 0$, then the Multiplicative group of integers modulo $P$ is *cyclic*.
  4. This cyclic group has a generator or primitive root modulo $P$, denoted with $g$, such that every number $a \in \mathbb{Z}_P$ can be written as $g^k \equiv a(\text{mod } P)$ for some integer $k$.

# Diffie-Hellman Example

Stony Brook University

- Example: Let p=11, g=2, then

| a | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $g^a$ | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512 | 1024 | 2048 |
| $g^a$ mod p | 2 | 4 | 8 | 5 | 10 | 9 | 7 | 3 | 6 | 1 | 2 |

A chooses 4, B chooses 3, then shared secret is $(2^3)^4 = (2^4)^3 = 2^{12} = 4 \pmod{11}$

**Adversaries sees $2^3=8$ and $2^4=5$, needs to solve one of $2^x=8$ and $2^y=5$ to figure out the shared secret.**

# DH and Hard Problems

Stony Brook University

- **Discrete Log (DLG) Problem**: Given <g, h, p>, computes a such that $g^a = h \bmod p$.
- **Computational Diffie Hellman (CDH) Problem**: Given <g, $g^a \bmod p$, $g^b \bmod p$> (without a, b) compute $g^{ab} \bmod p$.
- **Decision Diffie Hellman (DDH) Problem**: distinguish $(g^a, g^b, g^{ab})$ from $(g^a, g^b, g^c)$, where $a, b, c$ are randomly and independently chosen

# DH and Hard Problems

Stony Brook University

- **Discrete Log (DLG) Problem**: Given <g, h, p>, computes a such that $g^a = h \bmod p$.
- **Computational Diffie Hellman (CDH) Problem**: Given <g, $g^a \bmod p$, $g^b \bmod p$> (without a, b) compute $g^{ab} \bmod p$.
- **Decision Diffie Hellman (DDH) Problem**: distinguish $(g^a, g^b, g^{ab})$ from $(g^a, g^b, g^c)$, where $a, b, c$ are randomly and independently chosen

**1. If one can solve the DL problem, then one can solve the CDH problem.**

**2. If one can solve CDH, then one can solve DDH.**

# Assumptions

Stony Brook University

- **DDH Assumption**: *DDH is hard to solve*.
- **CDH Assumption**: *CDH is hard to solve*.
- **DLG Assumption**: *DLG is hard to solve*.

- ***DDH is assumed difficult to solve for large p in <u>classical computers</u>*** (e.g., at least 1024 bits).
- **Warning**:
  - New progress in 2013 can solve discrete log for p values with some properties. No immediate attack against practical setting yet.
  - Look out when you need to use/implement public key crypto
  - May want to consider Elliptic Curve-based algorithms, require shorter keys than discrete log (for 128-bit security require 256-bit key size)
  - Attacks on the use of DH in TLS: https://weakdh.org/

# ElGamal Encryption



- Public key <g, p, $h=g^a$ mod p>
- Private key is a
- **To encrypt**:
  - Chooses random b, computes C=[$g^b$ mod p, $g^{ab}$ * M mod p].
  - Idea: for each M, sender and receiver establish a shared secret $g^{ab}$ via the DH protocol. The value $g^{ab}$ hides the message M by multiplying it.
- **To decrypt** C=[$c_1$,$c_2$], computes M where
  - (($c_1^a$ mod p) * M) mod p = $c_2$.
    - To find M for x * M mod p = $c_2$, compute z s.t. x*z mod p =1, and then M = $C_2$*z mod p
- **CDH assumption ensures M cannot be fully recovered**
- **IND-CPA security requires DDH**
- **Multiplicatively homomorphic**

# Homomorphic Encryption

# Homomorphic Encryption

Stony Brook University

$E(m_1) \blacksquare E(m_2) = E(m_1 \odot m_2)$ [e.g., $\odot \in \{+,-,*,/\}$]

# Homomorphic Encryption

Stony Brook University

$$E(m_1) \blacksquare E(m_2) = E(m_1 \odot m_2) \text{ [e.g., } \odot \in \{+,-,*,/\}]$$

**Multiplicatively homomorphic:**

$E(m_1) \blacksquare E(m_2) = E(m_1 * m_2)$

**Additively homomorphic:**

$E(m_1) \blacksquare E(m_2) = E(m_1 + m_2)$

**Fully homomorphic:**

$E(m_1) \blacksquare E(m_2) = E(m_1 \odot m_2)$ [$\odot$ can be any operations]

# Homomorphic Encryption

Stony Brook University

$$E(m_1) \blacksquare E(m_2) = E(m_1 \odot m_2) \text{ [e.g., } \odot \in \{+,-,*,/\}]$$

**Multiplicatively homomorphic:**

$E(m_1) \blacksquare E(m_2) = E(m_1 * m_2)$

**Additively homomorphic:**

$E(m_1) \blacksquare E(m_2) = E(m_1 + m_2)$

**Fully homomorphic:**

$E(m_1) \blacksquare E(m_2) = E(m_1 \odot m_2)$ [$\odot$ can be any operations]

**Prove El-Gamal Public Key Encryption is multiplicatively homomorphic**

# RSA Algorithm

Stony Brook University

- Invented in **1978** by Ron **R**ivest, Adi **S**hamir and Leonard **A**dleman
  - Published as R L Rivest, A Shamir, L Adleman, "*On Digital Signatures and Public Key Cryptosystems*", Communications of the ACM, vol 21 no 2, pp120-126, Feb 1978
- **Security relies on the difficulty of factoring large composite numbers**
- Essentially the same algorithm was discovered in 1973 by Clifford Cocks, who works for the British intelligence
- **Multiplicatively homomorphic**

# Euler's Totient/Phi Function

Stony Brook University

$\varphi(n) =$ Number of positive intergers up to $n$ that are coprime/relatively prime
$\varphi(n) = |S_n|$ where $S_n = \{x \mid 1 \leq x \leq n \text{ and } gcd(x, n) = 1\}$

Suppose $n = P_1^{K_1} P_2^{K_2} \ldots P_r^{K_r}$ where each $P_i$ $(1 \leq i \leq r)$ is a unique prime that divides $n$. Then, $\varphi(n) = P_1^{k_1-1}(P_1 - 1)P_2^{k_2-1}(P_2 - 1)\ldots P_r^{k_r-1}(P_r - 1)$

If $n$ is a prime number then $\boxed{\varphi(n) = n - 1}$

If $n = p \times q$ where $p$ and $q$ are coprime then $\boxed{\varphi(n) = \varphi(p) \times \varphi(q) = (p-1)(q-1)}$

— END —

# RSA Public-Key Cryptosystem



**Key generation:**

1. Select 2 large prime numbers of about the same size, p and q

   Typically, each p, q has between 512 and 2048 bits

2. Compute $n = pq$, and $\Phi(n) = (q-1)(p-1)$
3. Select e, $1<e< \Phi(n)$, s.t. $\gcd(e, \Phi(n)) = 1$

   Typically, e=3 or e=65537

4. Compute d, $1< d< \Phi(n)$ s.t. $ed \equiv 1 \bmod \Phi(n)$

   Knowing $\Phi(n)$, d easy to compute.

**Public key: (e, n)**

**Private key: d**

# RSA Public-Key Cryptosystem



**Encryption:**

Given a message M such that $0 < M < n$ (or, $M \in Z_n - \{0\}$), to encrypt use the public key (e, n) to

compute $\mathbf{C = M^e \bmod n}$ where $C \in Z_n - \{0\}$

**Decryption:**

Given a ciphertext C, use the private key (d) to

compute $\mathbf{C^d \bmod n} = (M^e \bmod n)^d \bmod n = M^{ed} \bmod n = M$

# RSA Example



- $p = 11$, $q = 7$, $n = 77$, $\Phi(n) = 60$
- $d = 13$, $e = 37$ ($ed = 481$; $ed \bmod 60 = 1$)
- Let $M = 15$. Then $C \equiv M^e \bmod n$
  - $C \equiv 15^{37} \pmod{77} = 71$
- $M \equiv C^d \bmod n$
  - $M \equiv 71^{13} \pmod{77} = 15$

# RSA Example



- **Parameters:**
  - $p = 3$, $q = 5$, $n = pq = 15$
  - $\Phi(n) = ?$
- Let $e = 3$, what is d?
- **Given M=2, what is C?**
- **How to decrypt?**

# RSA Security and Hard Problems



$C = M^e \bmod (n=pq)$ →

Plaintext: M

Ciphertext: C

← $C^d \bmod n$

1. **Factoring Problem**: Given n=pq, compute p,q
2. **Finding RSA Private Key**: Given (n,e), compute d s.t. ed = 1 (mod $\Phi(n)$).
   - Given (d,e) such that ed = 1 (mod $\Phi(n)$), there is a clever randomized algorithm to factor n efficiently.
   - **Implication**: Cannot share the modulus n among multiple users
3. **RSA Problem**: From (n,e) and C, compute M s.t. $C = M^e$
   - Aka computing the e'th root of C.
   - Can be solved if n can be factored

# RSA Security and Factoring

Stony Brook University

- **Security depends on the difficulty of factoring n**
  - Factor n $\Rightarrow$ compute $\Phi(n)$ $\Rightarrow$ compute d from (e, n)
  - Knowing e, d such that ed = 1 (mod $\Phi(n)$) $\Rightarrow$ factor n
- **The length of n=pq reflects the strength**
  - 700-bit n factored in 2007
  - 768 bit n factored in 2009
- **RSA encryption/decryption speed is quadratic in key length**
- 1024 bit for minimal level of security today (roughly 80 bit security)
  - likely to be breakable in near future
- **Minimal 2048 bits recommended for current usage (roughly 112 bit security)**
- NIST suggests 15360-bit RSA keys are equivalent in strength to 256-bit
- **Factoring is easier to break with quantum computers**

# RSA Encryption and IND-CPA

Stony Brook University

- The RSA assumption, which assumes that the RSA problem is hard to solve, ensures that the plaintext cannot be fully recovered.

- **Plain RSA does not provide IND-CPA security**.
  - For Public Key systems, the adversary has the public key, hence the initial training phase is unnecessary, as the adversary can encrypt any message he wants to.

- **How to break IND-CPA security?**
- **How to use it more securely?**

# Real world usage of public key encryption



- **Often used to encrypt a symmetric key**
  - To encrypt a message M under an RSA public key (n,e), generate a new AES key K, compute [$K^e$ mod n, AES-$CBC_K$(M)]

- **Alternatively, one can use random padding.**
  - E.g., computer $(M \mid\mid r)^e$ mod n to encrypt a message M with a random value r
  - More generally, uses a function F(M,r), and encrypts as $F(M,r)^e$ mod n
  - From F(M,r), one should be able to recover M
  - This provides randomized encryption

# Digital Signature

Stony Brook University

- Consider the real-life example where a person pays by credit card and signs a bill; the seller verifies that the signature on the bill is the same with the signature on the card
- Contracts are valid if they are signed.
- Signatures provide **non-repudiation**.
  - Ensuring that a party in a dispute cannot repudiate or refute the validity of a statement or contract.
- Can we have a similar service in the electronic world?
  - **Does Message Authentication Code provide non-repudiation?**

# Digital Signature and Hash

- Very often digital signatures are used with hash functions, hash of a message is signed, instead of the message.

# Digital Signature and Hash

Stony Brook University

- Very often digital signatures are used with hash functions, hash of a message is signed, instead of the message.

**Why is the hash of a message signed instead of the whole message?**

# Digital Signature and Hash

Stony Brook University

- Very often digital signatures are used with hash functions, hash of a message is signed, instead of the message.

**Why is the hash of a message signed instead of the whole message?**

**Answer:**

**Public key schemes are expensive computationally**

# Digital Signature and Hash

Stony Brook University

- Very often digital signatures are used with hash functions, hash of a message is signed, instead of the message.

**Why is the hash of a message signed instead of the whole message?**

**Answer:**

**Public key schemes are expensive computationally**

**What properties are expected of the hash function and why?**

# Digital Signature and Hash

Stony Brook University

- Very often digital signatures are used with hash functions, hash of a message is signed, instead of the message.

**Why is the hash of a message signed instead of the whole message?**

**Answer:**
**Public key schemes are expensive computationally**

**What properties are expected of the hash function and why?**

**Answer:**
**Strong Collision Resistance**

# RSA Digital Signature

Stony Brook University

**Key generation (as in RSA encryption):**

- Select 2 large prime numbers of about the same size, p and q
- Compute $n = pq$, and $\Phi = (q - 1)(p - 1)$
- Select a random integer e, $1 < e < \Phi$, s.t. $\gcd(e, \Phi) = 1$
- Compute d, $1 < d < \Phi$ s.t. $ed \equiv 1 \bmod \Phi$

**Public key: (e, n)** **used for verification**

**Private key: d,** **used for generation**

# RSA Digital Signature

Stony Brook University

## Signing message M

- Verify $0 < M < n$
- Compute $S = h(M)^d \bmod n$

## Verifying signature S

- Use public key (e, n)
- Compute $S^e \bmod n = (h(M)^d \bmod n)^e \bmod n = h(M)$

# Non-repudiation

Stony Brook University

- Nonrepudiation is the assurance that someone cannot deny something.
- Typically, nonrepudiation refers to the ability to ensure that a party to a contract or a communication cannot deny the authenticity of their signature on a document or the sending of a message that they originated.

- Can one deny a digital signature one has made?

- Does email provide non-repudiation?

# Big Picture

Stony Brook University

| Properties | Symmetric key schemes | Asymmetric key schemes |
|---|---|---|
| Secrecy / Confidentiality | Stream ciphers<br>Block ciphers + encryption modes | Public key encryption: RSA, El Gamal, etc. |
| Authenticity / Integrity | Message Authentication Code | Digital Signatures: RSA, DSA, etc. |